

Computer Programming Education in the Age of Generative AI: Insights from Empirical Research

FITSUM GIZACHEW DERIBA*, School of Computing, University of Eastern Finland, Finland

ISMAILA TEMITAYO SANUSI*, School of Computing, University of Eastern Finland, Finland

OLADELE O. CAMPBELL*, Niger State Polytechnic, Nigeria

SOLOMON SUNDAY OYELERE*, Department of Computer Science, University of Exeter, United Kingdom

The integration of Generative Artificial Intelligence (GenAI) tools in programming education has become a focal point in recent years. This research delves into existing literature to explore the utilization of GenAI tools for teaching programming across different educational levels. Through a systematic review spanning from 2018 to 2023, we aim to offer insights into the current status of GenAI in programming education. Our analysis of 37 relevant studies indicates a predominant emphasis on higher education, constituting 81% of the reviewed literature. Among the findings, we identify six programming language instructions leveraging GenAI tools, with Python being the most prominent, featured in nearly half (49%) of the studies. The study also pinpoints primary GenAI applications such as Google's Bard (now Gemini), Codex, and Copilot, with ChatGPT emerging as the most commonly utilized tool for programming education. Objectives of the reviewed studies range from evaluating the performance of Large Language Models (LLMs) to understanding student and teacher perceptions regarding their usage. Additionally, these objectives encompass providing code explanation and suggestions, error correction, feedback provision, and integration with supportive approaches. Moreover, our analysis outlines significant learning outcomes associated with the adoption of GenAI in programming education, including increased motivation, interest, and engagement among students. Despite the potential benefits, the study identifies challenges faced by educators and students alike. Approximately 22% of the reviewed studies highlight various difficulties and concerns, ranging from issues with grading feedback to worries about student over-reliance on GenAI tools, which may lead to incorrect solutions and hinder learning progress. Overall, our study contributes to a comprehensive understanding of how GenAI tools are utilized in programming education. By analyzing the objectives and findings of the reviewed studies, we shed light on the evolving landscape of GenAI adoption and its implications for programming education.

Additional Key Words and Phrases: Computing Education, Computer Programming, Generative AI, Large Language Model, ChatGPT

ACM Reference Format:

Fitsum Gizachew Deriba, Ismaila Temitayo Sanusi, Oladele O. Campbell, and Solomon Sunday Oyelere. 2024. Computer Programming Education in the Age of Generative AI: Insights from Empirical Research . In . ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Since the advent of generative AI (GenAI) models such as ELMo, BERT, and OpenAI GPT in 2018, there has been an exponential increase in both the number of parameters and the capabilities of these tools [54]. GenAI models, designed to understand and generate human-like language, have sparked significant discussions within the scientific community about their applications and potential misuse[46]. The catalyst for these conversations was the sudden widespread availability of GenAI tools capable of generating and editing scientific texts, as well as answering scientific questions [6]. ChatGPT, for instance, can improve academic writing clarity, provide language support for non-native English writers, and address frequently asked questions [2, 15]. Consequently, the availability of GenAI models has prompted researchers and educators to investigate several open questions, initiating meaningful conversations and exploring these tools due to the continuous acceleration in their use.

The integration of GenAI into academia offers several opportunities and concerns. In computing education, these models are leveraged to address the skills gap of novice students in programming courses [25]. Examples include pair programming with AI collaboration and code explanation at different levels of abstraction [14]. Furthermore, real-time AI error suggestions and improvements in coding, as well as support on complex projects, present various opportunities for programmers [12].

The use of GenAI in programming education raises critical questions about pedagogy, creativity, and ethics [49]. Concerns include the risk of reduced proficiency in coding skills, potential diminishment of critical thinking abilities, and over-reliance on GenAI, which could hinder independent coding [23]. The paradoxical nature of utilizing GenAI in programming education

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Association for Computing Machinery.

Manuscript submitted to ACM

necessitates a thorough exploration to identify the boundaries of its use in computing education. Thus, we aim to review the current state of programming language education in the age of GenAI by posing the following questions:

RQ1. For what purpose is GenAI used in programming education?

RQ2. Which type of GenAI is widely adopted in programming education, and which programming languages are learned with GenAI?

RQ3. What are the main learning outcomes and challenges of using GenAI in programming education?

By answering these questions, the study aims to synthesize the current state of GenAI and programming education by exploring and providing insights from empirical studies. While we have identified existing reviews on GenAI use in education and other fields [4, 22], and notable contributions from ITiCSE working groups exploring the impact of AI and LLMs on computing education [45, 46], we have not found a systematic review of literature specifically on GenAI and programming education. An exception is a rapid review by Deriba et al. [16], who retrieved data only from Scopus and ACM and considered only articles published in 2023 for their review.

2 METHODOLOGY

In order to assess the impact of generative AI on programming education, we conducted a comprehensive literature review of recent studies. The review included an analysis of various research articles, conference papers, and preprints focusing on the integration and evaluation of AI tools such as ChatGPT, GPT-4, and GitHub Copilot in educational contexts. A total of 37 studies were identified and summarized in Table 1. These studies cover a wide range of topics, including the effectiveness of AI-generated solutions, the usability of AI tools by novice programmers, and the potential of large language models to enhance programming instruction. The diversity and breadth of these studies provide a robust foundation for understanding the current landscape and future directions of AI-assisted programming education. This review thoroughly examines studies by analyzing published papers on integrating GenAI in programming education for both learning and instructional purposes, adhering to the guidelines outlined by Kitchenham [26]. The entire review process, from defining the scope to the final curation of articles for in-depth analysis, follows a seven-step procedure as described by Siddaway [55].

2.1 Step 1: Definition of Review Scope and Research Questions

Building upon the research questions outlined in Section I, the first step focuses on developing keyword search strategies for relevant digital libraries. Four key libraries were chosen due to their extensive repositories in the field of computing and related areas: IEEE Xplore, Scopus, Web of Science, and ACM Digital Library [36].

2.2 Step 2: Developing Search Strategy

To capture all relevant studies pertaining to the research questions, a comprehensive search string was developed: ("Computing Education" OR "Programming") AND ("ChatGPT" OR "GenAI" OR "LLM" OR "Large Language Model" OR "Google Bard" OR "Copilot" OR "Bert" OR "Generative AI"). This search was conducted across the four databases from October 26 to 29, 2023. The initial retrieval yielded a total of 225 articles (Scopus: n=112, Web of Science: n=29, IEEE Xplore: n=46, and ACM Library: n=38). Given the large volume of results, a two-step filtering process was implemented. The first step utilized semi-automatic filters to apply pre-established exclusion and inclusion criteria, followed by a manual filtering process to identify the most relevant papers for further analysis [55].

2.3 Step 3: Inclusion and Exclusion Criteria

Before screening the studies, the first two authors established the following inclusion criteria: a) Studies should focus on programming education using GenAI and provide evidence of the learning or teaching setting. Theoretical discussions without evidence are excluded. b) The study should include original articles, conference proceedings, book chapters, and preprints to ensure the review encompasses the most current advancements in GenAI [46]. Editorials, reviews, opinions, reports, or commentary articles were excluded. c) Studies should be conducted in the English language. d) Studies should be published between 2018 and 2023.

2.4 Step 4: Removing Duplicate Documents

Duplicated studies were identified by importing the aggregated search results into Zotero, and duplicate records (n=58) were removed using the software's automatic feature. The remaining 167 studies were then imported into the Rayyan Systematic

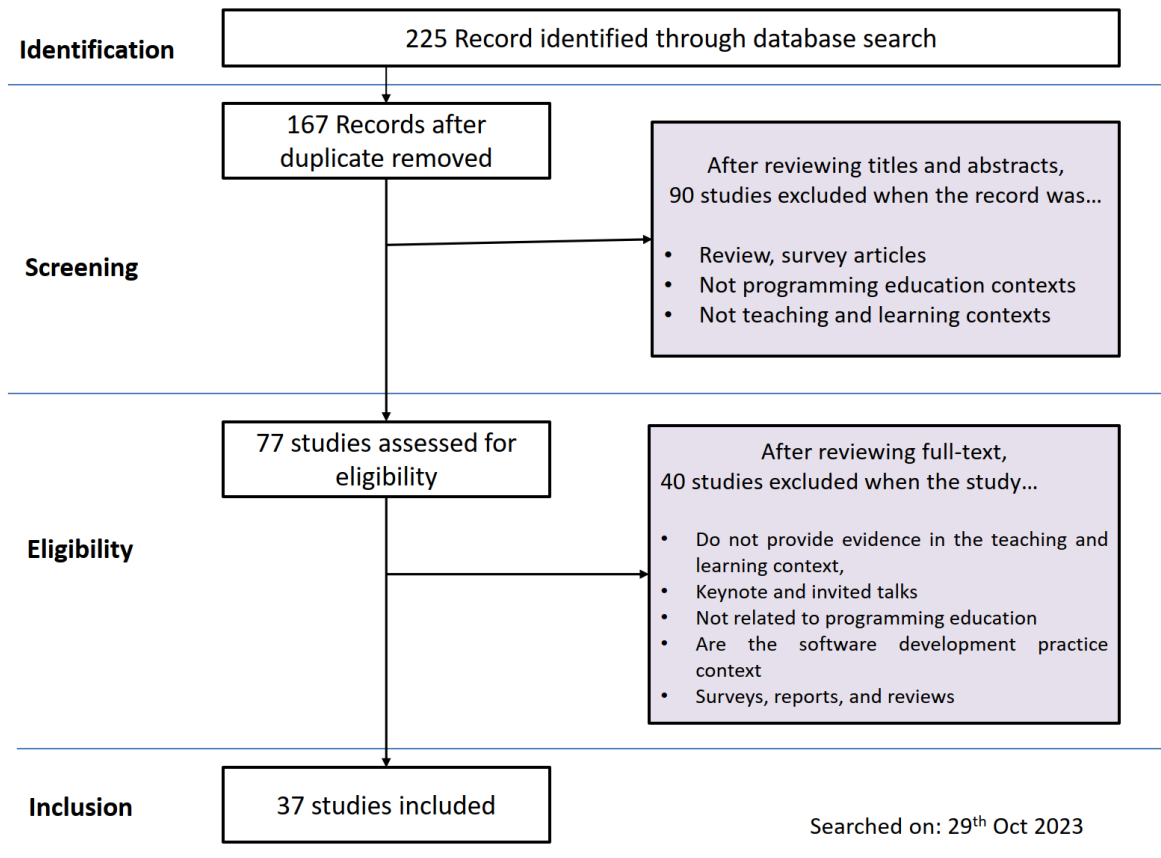


Fig. 1. Literature search and selection process

tool [40], a web application that supports the initial screening of abstracts and titles by two reviewers independently. The study selection and screening process are presented in Figure 1.

2.5 Step 5: Title and Abstract Screening

Imported studies underwent an initial screening based on their titles and abstracts. Researchers independently selected all articles discussing GenAI tools and programming education. Inter-rater agreement between the two researchers was 85%. Disagreements were resolved through discussion and consensus, resulting in 77 studies deemed eligible for full-text review.

2.6 Step 6: Full-Text Analysis and Coding

Before proceeding to full-text content analysis, the authors conducted a coding process based on the research questions [61]. This coding scheme categorized studies based on characteristics such as the country of study, education level, specific type of GenAI tool used, purpose of the tool, and programming language utilized. The authors also extracted data on learning outcomes and challenges encountered in teaching or learning programming with these tools.

2.7 Step 7: Further Exclusion and Refinement

Following the full-text analysis, a further selection process was undertaken based on the established inclusion criteria. Out of the initial 77 studies, 40 were excluded for various reasons: - Seventeen studies lacked evidence of application in a teaching and learning context. - Seven studies were identified as keynote or invited talks, which are not considered original research articles. - Six studies did not focus on programming education. - Five studies pertained to the broader field of software development practices rather than programming education specifically. - Five excluded studies were surveys, reports, or reviews, which do not align with the inclusion criteria for original research articles.

This refinement process resulted in a final selection of 37 studies that were deemed relevant and appropriate for further coding and analysis to answer the research questions guiding this investigation.

Table 1. Summary of Reviewed Studies by Publication Venue

SN	Publication Venue	Number of Studies	Reviewed Studies
1	Arxiv	5	[53], [52] [64], [42], [30]
2	ACM Conference on International Computing Education Research (ICER)	5	[56],[19] [34], [51], [29]
3	ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)	2	[31], [39]
4	IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)	3	[8], [5], [58]
5	ACM Technical Symposium on Computer Science Education	2	[32], [13]
6	ACM SIGPLAN International Symposium on SPLASH-E	1	[28]
7	ACM Transaction on Computer-Human Interaction	1	[47]
8	International Conference on Informatics in Schools. Cham: Springer Nature Switzerland	1	[59]
9	Journal of Systems and Software	1	[12]
10	Journal of Computing Sciences in Colleges	1	[48]
11	Systems and Information Engineering Design Symposium (SIEDS). IEEE	1	[1]
12	IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS)	1	[37]
13	IEEE/ACM International Workshop on Automated Program Repair. IEEE	1	[57]
14	CHI Conference on Human Factors in Computing Systems	1	[24]
15	ACM Inroads	1	[21]
16	Computers and Education: Artificial Intelligence	1	[63]
17	Artificial Intelligence in Education (AIED) Empowering Education with LLMs - the Next-Gen Interface and Content Generation Workshop	1	[20]
18	International Conference on Intelligent User Interfaces	1	[7]
19	International Conference on Artificial Intelligence in Education. Cham: Springer Nature Switzerland	2	[27], [9]
20	PLOS Computational Biology	1	[44]
21	IEEE. MIPRO ICT and Electronics Convention (MIPRO)	1	[3]
22	Australasian Computing Education Conference	2	[17], [18]
23	International Journal of Management	1	[43]
Total		37	

3 RESULT

3.1 Characteristics of Studies

The reviewed studies, sourced from 23 different venues, represent a broad spectrum of perspectives and research topics. They were published in reputable outlets such as *Arxiv*, ACM conferences and journals, IEEE conferences, Elsevier journals, and Springer conference proceedings. Out of the 37 studies reviewed, each venue contributed between 1 to 5 studies, with the highest number (5) found in both *Arxiv* and the ACM Conference on International Computing Education Research (ICER), as detailed in Table 1. This review encompasses studies published between 2022 (n=5) and 2023 (n=32). The studies span sixteen different countries (see Figure 2), with a focus on higher education (n=30, 81%). One study investigated K-8 education, while the remaining six studies (16%) did not specify the educational level.

3.2 Purposes of GenAI in Programming Education

he reviewed studies on the application of Generative AI (GenAI) in programming language instruction explore a multitude of dimensions. These diverse aspects are illustrated comprehensively in Figure 3.

3.2.1 Performance Evaluation of GenAI. Fifty-four percent of the reviewed studies focused on evaluating the performance of GenAI tools such as Copilot, Codex, GPT-3.0, GPT-3.5, and GPT-4 in various aspects of programming language education. These studies aimed to elucidate the roles that Generative AI models can play as digital tutors, assistants, and peers in the realm of programming language instruction. The evaluations encompassed a variety of scenarios, including program generation [64], grading feedback [43], pair programming [3], contextualized explanation, and task synthesis [43].

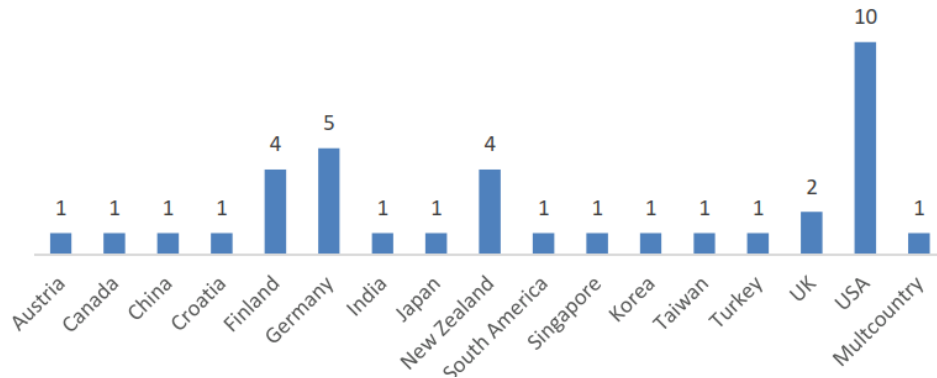


Fig. 2. Studies' geographic distribution

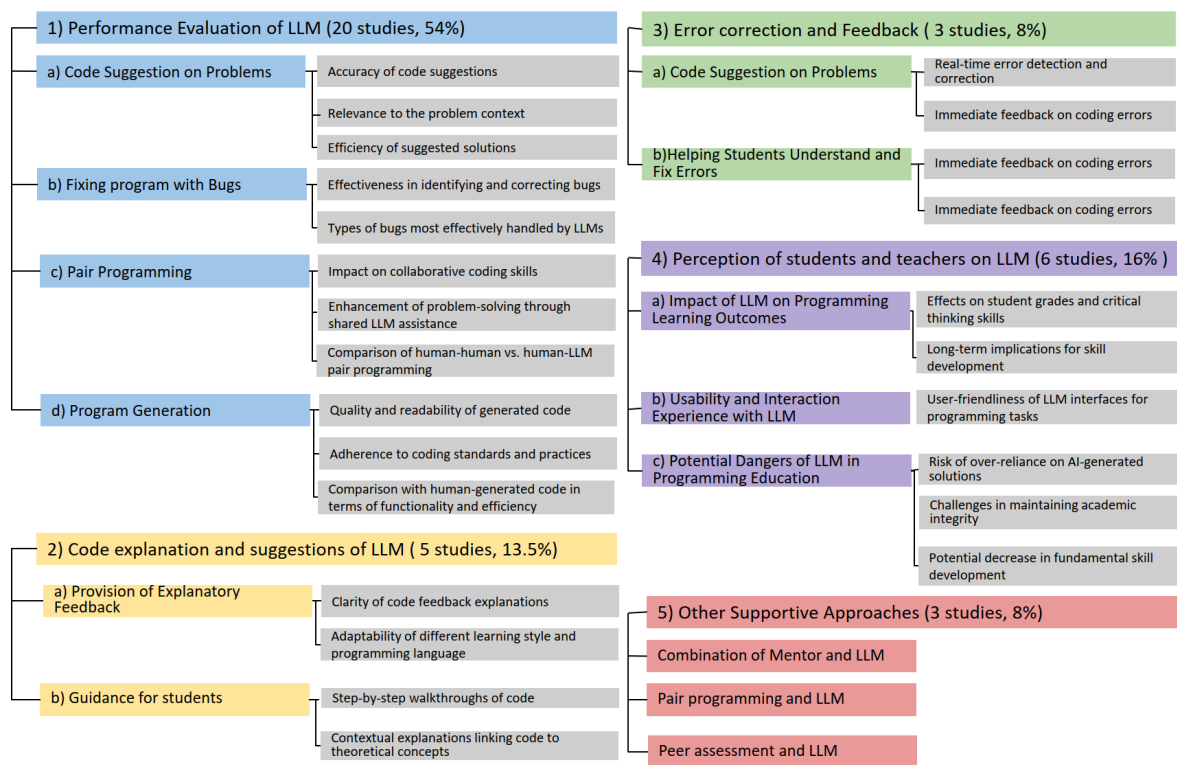


Fig. 3. Reasons for using GenAI in programming instruction

Studies also explored the effectiveness of GenAI tools, specifically GPT-3.5 and GPT-4, in enhancing computing education by providing code suggestions [39], handling different programming exercises and assessments [5, 53], and fixing programs with bugs [27, 57]. They compared code explanations generated by GenAI with those by students in terms of accuracy, length, and understandability [30], as well as interpretability and actionability [32]. Further evaluations involved assessing ChatGPT's effectiveness in generating coding solutions and its code understanding compared to human teachers across various error types [59].

Additionally, studies examined the capabilities of tools like Copilot and Codex in generating correct solutions for programming problems [12, 13, 17], providing code suggestions [48], and addressing challenges such as detecting plagiarism in AI-generated code [20]. They also explored the practicality of integrating GenAI into programming education [58], its capability to pass assessments [52], and automating the correction of programming mistakes using Automated Program Repair systems [64].

3.2.2 Perception of Students and Teachers on GenAI. Sixteen percent of the studies aimed to explore perceptions of GenAI tools among students and teachers. Studies examined instructors' plans to adapt courses in response to AI code generation tools [29], and the impact of GenAI on programming education methods [21]. They also investigated students' usability, interaction experience, and perceptions of benefits and risks [47], as well as novices' comprehension of AI-generated code [24]. Furthermore, studies explored students' perspectives on GenAI's impact on learning outcomes such as computational thinking, programming self-efficacy, motivation [63], and the potential of AI-enhanced gamification in teaching [7].

3.2.3 Code Explanation and Suggestions by GenAI. Thirteen and a half percent (13.5%) of the reviewed studies investigated the theme of code explanation and suggestions provided by GenAI tools. These studies aimed to uncover how GenAI applications such as ChatGPT, Codex, and others assist in generating code explanations and suggestions in programming education [9, 34]. They assessed the effectiveness of these tools in helping students understand code types and patterns across various programming languages, notably Python [44]. Evaluations focused on the sensibility, novelty, and practical applicability of the generated explanations [7]. Additionally, studies compared AI-generated code explanations with those generated by students to evaluate accuracy and comprehensibility [34].

3.2.4 Error Correction and Feedback. Eight percent (8%) of the studies focused on error correction and feedback mechanisms using GenAI in programming education. Tools like KOGI and integrated AI-assistance were employed to provide timely feedback and assistance to students encountering programming errors [28]. These studies explored how GenAI models analyze and provide feedback on syntax errors [42], runtime errors, and logical errors in programming exercises [19]. They aimed to enhance learning outcomes by automating error detection and correction processes.

3.2.5 Other Supportive Approaches. Eight percent (8%) of the studies investigated the integration of GenAI with other supportive approaches in programming education. These studies explored combinations such as mentorship with ChatGPT [37], pair programming with GenAI assistance [3], and ECM constructs with AI tools [8]. Goals included enhancing student motivation, improving learning outcomes, and developing effective programming language learning assistance systems. One study proposed an Exercise Creation Methodology to accurately label AI-generated answers for students' comprehension [8].

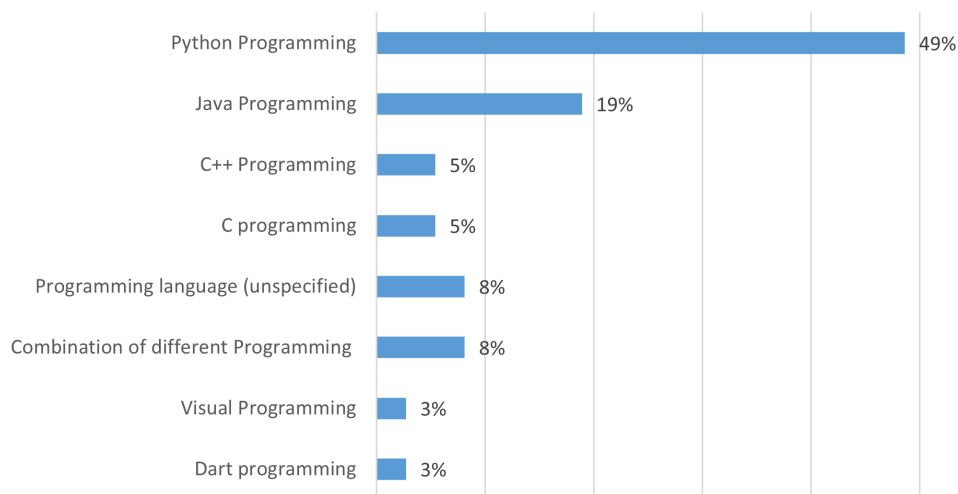


Fig. 4. Programming language classes employing GenAI tools

3.3 Programming Language Classes Leveraging GenAI Tools

The reviewed studies examined how different programming language instructions were integrated with GenAI tools, each tailored to specific educational goals. Figure 4 illustrates the distribution of programming languages used across the studies.

Python was the most frequently utilized programming language, employed in 49% of the studies, followed by Java (19%), C++ and C (both 5%), Dart and Visual Programming (3%) respectively, a combination of different programming languages (5%), and unspecified programming languages (8%).

The studies also identified various types of GenAI utilized in programming language instruction. ChatGPT emerged as the most prevalent, featured in 43% of the studies, encompassing versions such as GPT-3, GPT-3.5, GPT-4, and unspecified iterations. Codex was employed in 27% of the studies, while Copilot was used in 16%. Combinations of GenAI tools were also

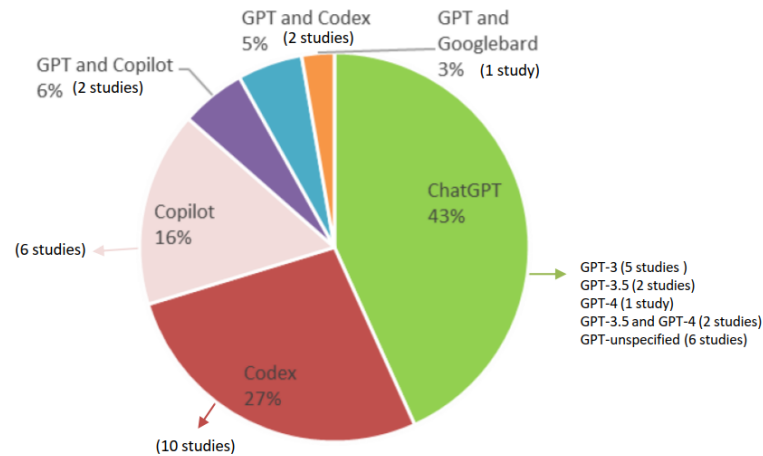


Fig. 5. GenAI tools employed in programming instruction

utilized in some studies, highlighting their versatility in programming education settings. Figure 5 provides an overview of the types of GenAI employed across the reviewed studies.

3.4 Learning Outcomes and Challenges of Using GenAI in Programming Language Education

This section identifies key learning outcomes associated with the use of GenAI in programming language education and explores challenges encountered in its implementation.

3.4.1 Motivation, Interest, and Engagement. Motivation, interest, and engagement emerge as primary learning outcomes in the context of using GenAI. Eight studies (22%) illustrate that GenAI tools, such as the Large Language Model (LLM), enhance student motivation and address programming challenges effectively. For instance, students exhibit heightened interest when GenAI identifies common errors like forgotten imports, typos, and spacing issues [28]. Kazemitabaar et al. [24] highlight that GenAI reduces stress and boosts engagement by improving task completion rates, code correctness, error reduction, and task efficiency. Jacques [21] demonstrate that diverse project representations, including pseudocode, enhance learner engagement by exploring various problem-solving approaches facilitated by GenAI.

Moreover, Cao [7] introduces a story-based gamification approach, significantly enhancing student motivation and engagement in programming courses. MacNeil et al. [34] emphasize that GenAI enriches student engagement through diverse explanations and code types, while Banic et al. [3] underscore GenAI's role in boosting interest and problem-solving skills among learners. Sarsa et al. [51] identify OpenAI Codex's proficiency in generating novel programming exercises, demonstrating high utility and effectiveness. Additionally, Phung et al. [42] introduce a model for precise feedback on syntax errors, enhancing educational outcomes.

3.4.2 Enhanced Personalized Learning Experience. Five studies (13.5%) emphasize GenAI's role in enhancing personalized learning experiences in programming education. One study reveals that GenAI improves learning by providing tailored explanations and identifying common mistakes, thereby promoting continuous skill improvement [34]. Another discusses integrating GenAI tools like GPTutor to offer personalized learning support, facilitating individualized learning paths for students [37].

Furthermore, an evaluation of Codex and GPT's performance highlights their effectiveness in enhancing personalized learning by addressing student queries in foundational programming courses [19]. Additionally, Copilot's pedagogical value is explored in helping students reflect on code failures and translate abstract concepts into practical solutions [13].

3.4.3 Clarity, Complexity, and Error Resolution in GenAI Solutions. Five additional studies focus on GenAI's impact on clarity, complexity management, and error resolution in programming languages. One study finds that GenAI-generated code explanations are as clear as human-generated ones, suggesting their effectiveness in communicating complex concepts clearly [30]. Another demonstrates Codex's ability to identify and rectify programming errors, though it faces challenges in certain programming categories such as strings [24]. Additionally, Codex's superiority over human performance in code writing and automatic repair tasks is highlighted [18].

Moreover, an evaluation of GPT and Google Bard reveals their effectiveness in iterative error correction processes, enhancing code quality and understanding in programming education [44].

3.4.4 Difficulty and Concerns in GenAI Implementation. Approximately 22% of reviewed studies identify challenges and concerns in implementing GenAI in programming education. Some studies note GenAI's limitations in grading feedback and task synthesis, especially in contexts involving ambiguous questions or multihop reasoning [43, 56]. Issues with GenAI providing inconsistent answers or struggling with certain coding scenarios, impacting its reliability, are also underscored [52]. Another discusses Codex's challenges in handling blank inputs and diverse test cases, posing difficulties for educators despite its potential benefits [17].

Concerns regarding student dependence on tools like Copilot, potentially hindering independent learning and critical thinking, are highlighted [47]. Additionally, assessment challenges, including inconsistent grading and misinterpretation of student code by GenAI tools, affecting educational outcomes, are explored [59].

3.4.5 Other Learning Outcomes of Using GenAI in Programming Languages. The remaining nine studies explore various aspects of GenAI's impact on programming education. Two studies propose strategies to prevent cheating by designing exams that challenge GenAI's capabilities, such as incorporating images and template codes [5, 29]. Another compares GenAI with human programmers, highlighting differences in solution diversity and accuracy [12]. Additionally, the effectiveness of machine learning algorithms in distinguishing GenAI-generated text for different prompts is examined [1].

One study applies Exercise Creation Methodology (ECM) with GenAI to facilitate programming exercise generation [8], while another evaluates code2vec's classification accuracy compared to GenAI-generated code [20].

These studies collectively illustrate GenAI's multifaceted impact on programming education, emphasizing both its potential benefits and the challenges that educators and learners must address.

4 DISCUSSION

4.1 Reflection of Findings

This study provides valuable insights into how GenAI has contributed to learning programming through the synthesis of primary studies. Our findings indicate that research on programming using GenAI has predominantly involved Higher Education Institution (HEI) students. This suggests an underrepresentation of K-12 students and adult learners in the existing knowledge base regarding GenAI's potential to support programming education for these groups. Additionally, the country representation of the reviewed articles reveals that no studies have been conducted within the African context. This underscores the need for future research to investigate how GenAI can assist students in learning programming in developing regions, where initiatives on programming education for young learners are limited [35, 41, 50].

This study also explores the purposes for which GenAI is used in programming education. Our findings reveal that most selected studies (54%) focus on evaluating the performance of Large Language Models (LLMs) in explaining programming concepts. This aligns with the ITiCSE working group report, where the majority of the reviewed papers fell into the same category, assessing the performance, capabilities, and limitations of LLMs in educational settings [46]. Evaluating GenAI's effectiveness in learning programming is essential, as educators and researchers seek approaches to facilitate learning across different categories of learners [16]. While the literature investigates the perceptions of educators and students regarding GenAI's use in programming classes, our results indicate that GenAI is used for code explanation, error correction, and integration with other assistive tools.

ChatGPT emerged as the most commonly used GenAI tool for supporting learning in programming, followed by Codex. Interestingly, we found that programming language tasks completed by GenAI tools were primarily based on Python programming. This finding supports the claim that Python is one of the fastest-growing programming languages [11, 38, 65].

This study also examines existing studies to understand the main learning outcomes and challenges of using GenAI tools in programming education. Our analyses reveal that GenAI helps to motivate and increase interest among users (educators and students), enhance personalized learning experiences, and clarify and explain coding tasks with clarity. Our results are consistent with prior research indicating that AI chatbots, including GenAI tools, support self-regulated learning [10], increase student engagement [33], and significantly boost learning interest [60]. However, we identified some challenges related to the use of GenAI, such as difficulties in grading feedback and providing incorrect assessments or solutions.

4.2 Limitations and Future Directions

While we explored databases that house publications within the fields of computer science and education, investigating more databases may yield more articles relevant to our analysis. This study considered only papers published in English, suggesting that some related papers may have been omitted due to our exclusion criteria. Our findings suggest that limited work has focused on K-12 students, adult learners, and developing regions of Africa. Future research should investigate how young learners can be supported in learning programming with GenAI, while also considering adult education. Future work should also examine the capabilities of GenAI to assist learners in the African context, including comparative studies with developed regions to address the limited work on computing education research (CER) from the African context. We could not identify any studies exploring the use of GenAI as a tool to address gender differences in programming education, including learning ability. Researchers should investigate whether GenAI can foster programming and computing learning in both girls and boys.

5 CONCLUSION

Through a systematic literature review, this study investigates the effect of GenAI on learning programming. Using 37 selected studies, we provided insights into the purposes for which GenAI is used in programming education, the types of GenAI adopted in programming education, and the main learning outcomes, including the challenges of using GenAI in programming education. We believe this study contributes to the knowledge of how GenAI has been used to support students, educators, and researchers in enhancing the learning of programming. Since teaching and learning programming have been characterized by problems related to language feature complexity, especially for novices, strategic approaches must be promoted. According to Xinogalos [62], "*selecting a strategy for an initial approach to teaching programming is the first decision that has to be made,*" which in this case can be the use of GenAI tools.

REFERENCES

- [1] Hosam Alamlah, Ali Abdullah S AlQahtani, and AbdElRahman ElSaid. 2023. Distinguishing Human-Written and ChatGPT-Generated Text Using Machine Learning. In *2023 Systems and Information Engineering Design Symposium (SIEDS)*. IEEE, 154–158.
- [2] Yazid Albadarin, Mohammed Sagr, Nicolas Pope, and Markku Tukiainen. 2024. A systematic literature review of empirical research on ChatGPT in education. *Discover Education* 3, 1 (2024), 60.
- [3] Bojan Banić, Mario Konecki, and Mladen Konecki. 2023. Pair programming education aided by ChatGPT. In *2023 46th MIPRO ICT and Electronics Convention (MIPRO)*. IEEE, 911–915.
- [4] Peter Bannister, Alexandra Santamaría Urbieto, and Elena Alcalde Peñalver. 2023. A systematic review of generative AI and (english medium instruction) higher education. *Aula Abierta* 52, 4 (2023), 401–409.
- [5] Jonnathan Berrezueta-Guzman and Stephan Krusche. 2023. Recommendations to create programming exercises to overcome ChatGPT. In *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 147–151.
- [6] Abeba Birhane, Atoosa Kasirzadeh, David Leslie, and Sandra Wachter. 2023. Science in the age of large language models. *Nature Reviews Physics* (2023), 1–4.
- [7] Chen Cao. 2023. Scaffolding CS1 Courses with a Large Language Model-Powered Intelligent Tutoring System. In *Companion Proceedings of the 28th International Conference on Intelligent User Interfaces*. 229–232.
- [8] WK Chan, YT Yu, Jacky W Keung, and Victor CS Lee. 2023. Toward AI-assisted Exercise Creation for First Course in Programming through Adversarial Examples of AI Models. In *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 132–136.
- [9] Eason Chen, Ray Huang, Han-Shin Chen, Yuen-Hsien Tseng, and Liang-Yi Li. 2023. GPTutor: a ChatGPT-powered programming tool for code explanation. In *International Conference on Artificial Intelligence in Education*. Springer, 321–327.
- [10] Thomas KF Chiu. 2023. The impact of Generative AI (GenAI) on practices, policies and research direction in education: A case of ChatGPT and Midjourney. *Interactive Learning Environments* (2023), 1–17.
- [11] Oluwaseun Alexander Dada, George Obaido, Ismaila Temitayo Sanusi, Kehinde Aruleba, and Abdullahi Abubakar Yunusa. 2022. Hidden gold for it professionals, educators, and students: Insights from stack overflow survey. *IEEE Transactions on Computational Social Systems* 10, 2 (2022), 795–806.
- [12] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [13] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 1136–1142.
- [14] Paul Denny, James Prather, Brett A Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing education in the era of generative AI. *Commun. ACM* 67, 2 (2024), 56–67.
- [15] Ismail Dergaa, Karim Chamari, Piotr Zmijewski, and Helmi Ben Saad. 2023. From human writing to artificial intelligence generated text: examining the prospects and potential threats of ChatGPT in academic writing. *Biology of Sport* 40, 2 (2023), 615–622.
- [16] Fitsum Gizachew Deriba, Ismaila Temitayo Sanusi, and Amos Oyelere Sunday. 2023. Enhancing Computer Programming Education using ChatGPT: A Mini Review. In *23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. ACM.
- [17] James Finnie-Ansley, Paul Denny, Brett A Becker, Andrew Luxton-Reilly, and James Prather. 2022. The robots are coming: Exploring the implications of openai codex on introductory programming. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19.
- [18] James Finnie-Ansley, Paul Denny, Andrew Luxton-Reilly, Eddie Antonio Santos, James Prather, and Brett A Becker. 2023. My AI Wants to Know if This Will Be on the Exam: Testing OpenAI's Codex on CS2 Programming Exercises. In *Proceedings of the 25th Australasian Computing Education Conference*. 97–104.
- [19] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutchme, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the Responses of Large Language Models to Beginner Programmers' Help Requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1*. ACM,

- 93–105.
- [20] Muntasir Hoq, Yang Shi, Juho Leinonen, Damilola Babalola, Collin Lynch, and Bitu Akram. 2023. Detecting ChatGPT-Generated Code in a CS1 Course. In *AIED2023 Empowering Education with LLMs - the Next-Gen Interface and Content Generation Workshop* (03–07). Tokyo, Japan.
- [21] Lorraine Jacques. 2023. Teaching CS-101 at the Dawn of ChatGPT. *ACM Inroads* 14, 2 (2023), 40–46.
- [22] Athanasios Karapantelakis, Pegah Alizadeh, Abdulrahman Alabassi, Kaushik Dey, and Alexandros Nikou. 2023. Generative AI in mobile networks: a survey. *Annals of Telecommunications* (2023), 1–19.
- [23] Enkelejd Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences* 103 (2023), 102274.
- [24] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–23.
- [25] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2023. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. 1–12.
- [26] Barbara Kitchenham and Pearl Brereton. 2013. A systematic review of systematic review process research in software engineering. *Information and software technology* 55, 12 (2013), 2049–2075.
- [27] Charles Koutchme, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2023. Automated Program Repair Using Generative Models for Code Infilling. In *International Conference on Artificial Intelligence in Education*. Springer, 798–803.
- [28] Kimio Kuramitsu, Yui Obara, Miyu Sato, and Momoka Obara. 2023. KOGI: A Seamless Integration of ChatGPT into Jupyter Environments for Programming Education. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E*. 50–59.
- [29] Sam Lau and Philip Guo. 2023. From "Ban it till we understand it" to "Resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1*. 106–121.
- [30] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing code explanations created by students and large language models. *arXiv preprint arXiv:2304.03938* (2023).
- [31] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing Code Explanations Created by Students and Large Language Models. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (Turku, Finland) (*ITI/CSE 2023*). Association for Computing Machinery, New York, NY, USA, 124–130. <https://doi.org/10.1145/3587102.3588785>
- [32] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A Becker. 2023. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 563–569.
- [33] Chen-Chung Liu, Mo-Gang Liao, Chia-Hui Chang, and Hung-Ming Lin. 2022. An analysis of children's interaction with an AI chatbot and its impact on their interest in reading. *Computers & Education* 189 (2022), 104576.
- [34] Stephen MacNeil, Andrew Tran, Dan Mogil, Seth Bernstein, Erin Ross, and Ziheng Huang. 2022. Generating diverse code explanations using the gpt-3 large language model. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 2*. 37–39.
- [35] Monica M McGill, Leigh Ann DeLyster, Ismaila Temitayo Sanusi, and Selina Marianna Shah. 2023. Meeting the Needs of All Learners through High Quality K-12 Computing Education Research. In *Proceedings of the ACM Conference on Global Computing Education Vol 2*. 185–186.
- [36] Philippe Mongeon and Adèle Paul-Hus. 2016. The journal coverage of Web of Science and Scopus: a comparative analysis. *Scientometrics* 106 (2016), 213–228.
- [37] JunSeong Moon, RaeEun Yang, SoMin Cha, and Seong Baeg Kim. 2023. chatGPT vs Mentor: Programming Language Learning Assistance System for Beginners. In *2023 IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS)*. IEEE, 106–110.
- [38] Abhinav Nagpal and Goldie Gabrani. 2019. Python for data analytics, scientific and technical applications. In *2019 Amity international conference on artificial intelligence (AICAI)*. IEEE, 140–145.
- [39] Eng Lieh Ouh, Benjamin Kok Siew Gan, Kyong Jin Shim, and Swavek Wlodkowski. 2023. ChatGPT, Can You Generate Solutions for my Coding Exercises? An Evaluation on its Effectiveness in an Undergraduate Java Programming Course. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 54–60.
- [40] Mourad Ouzzani, Hossam Hammady, Zbys Fedorowicz, and Ahmed Elmagarmid. 2016. Rayyan—a web and mobile app for systematic reviews. *Systematic reviews* 5 (2016), 1–10.
- [41] Mayowa Oyedoyin, Ismaila Temitayo Sanusi, and Musa Adekunle Ayanwale. 2024. Young children's conceptions of computing in an African setting. *Computer Science Education* (2024), 1–36.
- [42] Tung Phung, José Cambrero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating High-Precision Feedback for Programming Syntax Errors using Large Language Models. *arXiv preprint arXiv:2302.04662* (2023).
- [43] Tung Phung, Victor-Alexandru Pădurean, José Cambrero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generative AI for Programming Education: Benchmarking ChatGPT, GPT-4, and Human Tutors. *International Journal of Management* 21, 2 (2023), 100790.
- [44] Stephen R Piccolo, Paul Denny, Andrew Luxton-Reilly, Samuel H Payne, and Perry G Ridge. 2023. Evaluating a large language model's ability to solve programming exercises from an introductory bioinformatics course. *PLOS Computational Biology* 19, 9 (2023), e1011511.
- [45] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michael E. Caspersen, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. Transformed by Transformers: Navigating the AI Coding Revolution for Computing Education: An ITiCSE Working Group Conducted by Humans. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 2* (Turku, Finland) (*ITI/CSE 2023*). Association for Computing Machinery, New York, NY, USA, 561–562. <https://doi.org/10.1145/3587103.3594206>
- [46] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education* (Turku, Finland) (*ITI/CSE-WGR '23*). Association for Computing Machinery, New York, NY, USA, 108–159. <https://doi.org/10.1145/3623762.3633499>
- [47] James Prather, Brent N Reeves, Paul Denny, Brett A Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That It Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *ACM Transactions on Computer-Human Interaction* 31, 1 (2023), 1–31.
- [48] Ben Puryear and Gina Sprint. 2022. Github copilot in the classroom: learning to code with AI assistance. *Journal of Computing Sciences in Colleges* 38, 1 (2022), 37–47.

- [49] Partha Pratim Ray. 2023. ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. *Internet of Things and Cyber-Physical Systems* (2023).
- [50] Ismaila Temitayo Sanusi and Fitsum Deriba. 2024. What do we know about Computing Education in Africa? A Systematic Review of Computing Education Research Literature. In *arXiv preprint*.
- [51] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1*. 27–43.
- [52] Jaromir Savelka, Arav Agarwal, Marshall An, Chris Bogart, and Majd Sakr. 2023. Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses. *arXiv preprint arXiv:2306.10073* (2023).
- [53] Jaromir Savelka, Arav Agarwal, Christopher Bogart, and Majd Sakr. 2023. Large language models (gpt) struggle to answer multiple-choice questions about code. *arXiv preprint arXiv:2303.08033* (2023).
- [54] Abigail See, Aneesh Pappu, Rohun Saxena, Akhila Yerukola, and Christopher D Manning. 2019. Do massively pretrained language models make better storytellers? *arXiv preprint arXiv:1909.10705* (2019).
- [55] Andy P Siddaway, Alex M Wood, and Larry V Hedges. 2019. How to do a systematic review: a best practice guide for conducting and reporting narrative reviews, meta-analyses, and meta-syntheses. *Annual review of psychology* 70 (2019), 747–770.
- [56] Adish Singla. 2023. Evaluating ChatGPT and GPT-4 for Visual Programming. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 2*. 14–15.
- [57] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of ChatGPT. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 23–30.
- [58] Sandro Speth, Niklas Meißner, and Steffen Becker. 2023. Investigating the Use of AI-Generated Exercises for Beginner and Intermediate Programming Courses: A ChatGPT Case Study. In *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 142–146.
- [59] Markus Wieser, Klaus Schöffmann, Daniela Stefanics, Andreas Bollin, and Stefan Pasterk. 2023. Investigating the Role of ChatGPT in Supporting Text-Based Programming Education for Students and Teachers. In *International Conference on Informatics in Schools: Situation, Evolution, and Perspectives*. Springer Nature Switzerland Cham, 40–53.
- [60] Rong Wu and Zhonggen Yu. 2023. Do AI chatbots improve students learning outcomes? Evidence from a meta-analysis. *British Journal of Educational Technology* (2023).
- [61] Betelhem Zewdu Wubineh, Fitsum Gizachew Deriba, and Michael Melese Woldeyohannis. 2023. Exploring the opportunities and challenges of implementing artificial intelligence in healthcare: A systematic literature review. In *Urologic Oncology: Seminars and Original Investigations*. Elsevier.
- [62] Stelios Xinogalos. 2016. Designing and deploying programming courses: Strategies, tools, difficulties and pedagogy. *Education and Information Technologies* 21 (2016), 559–588.
- [63] Ramazan Yilmaz and Fatma Gizem Karaoglan Yilmaz. 2023. The effect of generative artificial intelligence (AI)-based tool use on students’ computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence* (2023), 100147.
- [64] Jialu Zhang, José Cambroner, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).
- [65] Zhaoxu Zhang, Hengcheng Zhu, Ming Wen, Yida Tao, Yepang Liu, and Yingfei Xiong. 2020. How do python framework apis evolve? an exploratory study. In *2020 IEEE 27th international conference on software analysis, evolution and reengineering (saner)*. IEEE, 81–92.